# Detailed Code Explanation: 3DGS to TLS Point Cloud Alignment Tool

---

## Purpose & Context

This script solves a core problem in forest scanning research: **3D Gaussian Splatting (3DGS)** produces point clouds from photos, but they exist in an arbitrary, unitless coordinate frame. **Terrestrial Laser Scanning (TLS)** produces point clouds in real-world metric coordinates (meters). To compute Leaf Area Index (LAI) and other canopy measurements, you must bring the 3DGS output into the same scale and coordinate frame as the TLS reference. This script does exactly that — it computes a similarity transform (scale `s`, rotation `R`, translation `t`) and applies it to every attribute of a 3DGS PLY file without losing a single byte of data.

---

## Imports

```python
import os, sys, argparse
import numpy as np
from plyfile import PlyData, PlyElement
```

- `os` / `sys`: path operations and CLI argument manipulation.
- `argparse`: parses command-line flags.
- `numpy`: all linear algebra (matrices, SVD, norms).
- `plyfile`: reads/writes PLY files preserving every attribute field (positions, colors, spherical harmonics, opacities, scales, rotations). The key advantage over Open3D for reading is **zero data loss** — Open3D would discard 3DGS-specific attributes.

---

## Section 1: Mathematical Utility Functions

### 1.1 `umeyama(X, Y)` — Optimal Similarity Transform via SVD

**Goal:** Given N corresponding point pairs — source points `X` (from 3DGS) and reference points `Y` (from TLS) — find the scalar `s`, rotation matrix `R`, and translation vector `t` that minimise the least-squares alignment error:

```
Y ≈ s · R · X + t
```

This is the **Umeyama (1991)** closed-form solution.

**Step-by-step math:**

python

```
n, m = X.shape          # n = number of points, m = 3 (dimensions)
mu_x = X.mean(axis=0)   # centroid of source:    μ_x ∈ ℝ³
mu_y = Y.mean(axis=0)   # centroid of reference: μ_y ∈ ℝ³
X_centered = X - mu_x   # zero-mean source
Y_centered = Y - mu_y   # zero-mean reference
```

**Variance of source points** (average squared distance from centroid):

```
σ²_x = (1/n) · Σᵢ ||xᵢ - μ_x||²
```

This normalises the scale calculation so it doesn't depend on how spread out X is.

**Cross-covariance matrix** $\Sigma \in \mathbb{R}^{3X3}$:

```
Σ = (1/n) · Y_centered.T @ X_centered
```

This matrix captures how Y's spread correlates with X's spread in each axis combination.

**Singular Value Decomposition:**

```
U, D, Vt = SVD(Σ)     →    Σ = U · diag(D) · Vt
```

U and Vt are orthogonal matrices (rotations/reflections). D is a 1D array of singular values (non-negative, sorted descending).

**Reflection fix** — SVD can produce a reflection (det = -1) instead of a pure rotation. The correction matrix (S) negates the last diagonal element if needed:

python

```
S = np.eye(m)
if det(U) * det(Vt) < 0:
    S[m-1, m-1] = -1
```

This ensures `det(R) = +1` (proper rotation, not a mirror).

**Optimal rotation:**

$$R = U \cdot S \cdot Vt$$

**Optimal scale** — sum of signed singular values divided by source variance:

$$s = \Sigma_i \, D_i_signed \; / \; \sigma^2_x$$

The `D_signed` vector has its last element negated if there was a reflection. Geometrically, `s` is how much larger the TLS cloud is relative to the 3DGS cloud (typically > 1 since 3DGS has no absolute scale).

**Optimal translation:**

$$t = \mu_y - s \cdot R \cdot \mu_x$$

This ensures that after applying `s·R·X + t`, the centroid of the transformed source aligns with the centroid of the reference.

**Return:** `(s, R, t)` — the three parameters of the similarity transform.

---

### 1.2 `rotation_matrix_to_quaternion(R)` — Sheppard's Method

A rotation matrix `R` (3×3) can be represented as a **unit quaternion** `q = (w, x, y, z)` where `w` is the scalar part and `(x, y, z)` is the vector part. Quaternions are compact (4 floats vs 9) and numerically stable for composition.

The relationship between `R` and `q` is:

$$R = (2w^2 - 1)\cdot I + 2\cdot[vv^T] + 2w\cdot[v\times]$$

where `v = (x, y, z)` and `[v×]` is the skew-symmetric cross-product matrix.

**Sheppard's method** avoids near-zero denominators by choosing which quaternion component to compute first based on which is largest. The trace of R is:

$$tr(R) = R_{00} + R_{11} + R_{22} = 4w^2 - 1$$

**Case 1:** `tr(R) > 0` → `w` is the largest component.

$$
\begin{aligned}
S &= 2 \cdot \sqrt{(\mathrm{tr}(R) + 1)} \\
w &= S/4 \\
x &= (R_{21} - R_{12}) / S \\
y &= (R_{02} - R_{20}) / S \\
z &= (R_{10} - R_{01}) / S
\end{aligned}
$$

**Cases 2, 3, 4** handle when one of `x`, `y`, or `z` is dominant (when the corresponding diagonal element is largest). The formulas are analogous, solving for that component first.

Finally, the quaternion is **normalised** to unit length: `q / ||q||`. This removes floating-point drift.

---

### 1.3 `multiply_quaternions(q1, q2)` — Batch Quaternion Multiplication

In 3DGS, each Gaussian splat has an orientation stored as a quaternion `(rot_0, rot_1, rot_2, rot_3)` = `(w, x, y, z)`. When we rotate the entire scene by rotation matrix `R` (converted to quaternion `q_R`), each Gaussian's orientation must be updated to `q_R ⊗ q_old` (quaternion product).

`q1` is the scene rotation quaternion — shape `(4,)`. `q2` is the array of all Gaussian quaternions — shape `(N, 4)`.

**Quaternion multiplication formula** (Hamilton product):

$$
\begin{aligned}
&q_1 \otimes q_2: \\
w &= w_1 \cdot w_2 - x_1 \cdot x_2 - y_1 \cdot y_2 - z_1 \cdot z_2 \\
x &= w_1 \cdot x_2 + x_1 \cdot w_2 + y_1 \cdot z_2 - z_1 \cdot y_2 \\
y &= w_1 \cdot y_2 - x_1 \cdot z_2 + y_1 \cdot w_2 + z_1 \cdot x_2 \\
z &= w_1 \cdot z_2 + x_1 \cdot y_2 - y_1 \cdot x_2 + z_1 \cdot w_2
\end{aligned}
$$

This is applied vectorised using NumPy broadcasting — `q2[..., 0]` extracts the w-component of all N quaternions simultaneously.

The result is **re-normalised** to unit length to prevent cumulative drift.

**Why left-multiply by `q_R`?** Because we're composing "first apply the Gaussian's own orientation, then rotate by `q_R`". Left multiplication means `q_R` is applied last in world space, which is what we want when rotating the whole scene.

---

## Section 2: Main Alignment Logic

### 2.1 `get_bbox_transform(ref_bbox, src_bbox)` — Bounding Box Alignment

This computes a rough scale and translation by matching the bounding boxes of the two point clouds. It accepts either a 4-element (x, y only) or 6-element (x, y, z) bounding box.

```
scale_x = (ref_xmax - ref_xmin) / (src_xmax - src_xmin)
scale_y = ref_dy / src_dy
scale_z = ref_dz / src_dz   (if 3D)
s = average(scale_x, scale_y, scale_z)
```

This is an **isotropic** scale — one number applies to all axes. It uses the average of axis-wise scales as a compromise.

The translation is derived from matching centres:

```
ref_center = midpoint of reference bbox
src_center = midpoint of source bbox
t = ref_center - s · src_center
```

**Rotation is Identity (no rotation)** because bounding boxes carry no orientation information. This is the weakest alignment method but useful as a first pass.

When only 2D bbox is given, the Z translation is set to zero — the tool makes no assumption about height alignment.

---

### 2.2 `compute_bbox_from_pts(pts)` — Simple Bounding Box

Takes a `(N, 3)` numpy array of points and returns `[xmin, xmax, ymin, ymax, zmin, zmax]` using `np.min` and `np.max` per axis. No mathematical complexity — just array extrema.

---

### 2.3 `extract_main_cluster(pcd, voxel_size)` — Robust Cluster Extraction

The problem: both 3DGS and TLS point clouds often contain noise, floating outliers, and background points unrelated to the tree. Computing a bounding box over all points gives a wildly incorrect scale. This function isolates the dominant object (the tree).

**Step 1 — Voxel Downsampling:** Divides space into voxels of size `voxel_size` meters. Keeps one point per voxel (the centroid). This drastically reduces point count for fast processing without losing the shape.

**Step 2 — Statistical Outlier Removal:** For each point, computes the mean distance to its `nb_neighbors=20` nearest neighbours. Points whose distance is more than `std_ratio=2.0` standard deviations above the global mean are removed. This kills isolated noise points.

**Step 3 — DBSCAN Clustering:** DBSCAN (Density-Based Spatial Clustering of Applications with Noise) groups nearby points into clusters without requiring the number of clusters to be known in advance.

Parameters:

- `eps = voxel_size * 5` — two points are neighbours if they are within `eps` metres of each other.
- `min_points = 10` — a cluster must have at least 10 points; otherwise points are labelled as noise (-1).

The output `labels` array assigns each point to a cluster ID (0, 1, 2, ...) or -1 for noise.

```python
counts = np.bincount(labels[labels >= 0])
largest_cluster_idx = counts.argmax()
```

`bincount` counts how many points belong to each cluster ID. `argmax` returns the ID with the most points — this is the tree canopy, which dominates the scene. The function returns only the points belonging to this cluster.

---

### 2.4 `get_pcd_bbox(pcd)` — Open3D PointCloud Bounding Box

Converts Open3D PointCloud to numpy via `np.asarray(pcd.points)` then computes axis extrema. Returns `[xmin, xmax, ymin, ymax, zmin, zmax]`. Guards against empty clouds.

---

### 2.5 `pick_points(pcd, title)` — Interactive 3D Point Picking

Opens an Open3D `VisualizerWithEditing` window. The user rotates the view with left-drag and Shift+clicks to select specific 3D points. After closing the window, `vis.get_picked_points()` returns the indices of selected points.

The selected indices are used to extract `(x, y, z)` coordinates from `pcd.points`. These are then passed to `umeyama()` to compute the exact similarity transform. Requires at least 3 points (3 non-collinear point pairs fully constrain a 3D similarity transform: scale has 1 DOF, rotation has 3, translation has 3 = 7 total; 3 points give 9 equations which over-constrains the system nicely).

---

### 2.6 `auto_align_z_sweep(ref_pcd, src_pcd, voxel_size)` — Rotation Search

After bounding-box alignment, the 3DGS cloud may be correctly scaled and centred but rotated about the vertical (Z) axis — because 3DGS has no canonical "north" direction.

This function sweeps through all **36 candidate Z-rotations** in 10-degree steps (0°, 10°, 20°, ..., 350°) and finds which one gives the best ICP alignment score.

For each angle θ:

```python
T_init = [[cosθ, -sinθ, 0, 0],
          [sinθ,  cosθ, 0, 0],
          [0,     0,    1, 0],
          [0,     0,    0, 1]]
```

This is a pure Z-axis rotation matrix embedded in a 4×4 homogeneous transform.

ICP (see below) is run with only 30 iterations and a tight `max_corr_dist = voxel_size * 1.5`. The fitness score (fraction of source points with a corresponding reference point within `max_corr_dist`) is compared across all angles. The `T_final` from the best-fitness angle is returned and used as the initial transform for the full ICP refinement.

**Why only Z?** In forest scanning, the tree is always upright. TLS and 3DGS both capture it vertically. The only unknown angular DOF is the azimuthal rotation around Z.

---

### 2.7 `run_icp_alignment(ref_pts, src_pts, init_s, init_R, init_t, ...)` — ICP Refinement

**ICP (Iterative Closest Point)** is an algorithm that iteratively:

1. For each source point, finds the nearest reference point.
2. Computes the optimal rigid transform to minimise mean-square distance to matched pairs.

3. Applies the transform and repeats.

This converges to a local minimum. It needs a good initialisation (hence the bounding-box/Umeyama step first).

**Critical pre-scaling step:**

```python
src_pcd_down.scale(init_s, center=(0,0,0))
```

Open3D's ICP expects the initial transform `T_init` to be a **rigid** 4×4 matrix (no scale). If you embed scale in T_init, the Point-to-Plane ICP solver produces numerically unstable results. So the source cloud is pre-scaled in-place, and `T_init` contains only rotation and translation.

**Normal estimation:** Normals are computed for Point-to-Plane ICP, which typically converges faster and more accurately than Point-to-Point. For each point, a local plane is fit to its `max_nn=30` nearest neighbours within `radius = voxel_size * 2` metres. The residual for each point pair is measured as the distance along the surface normal rather than Euclidean distance — this is better for smooth surfaces like tree bark.

**Global Registration (optional, `--global_reg`):** Instead of using bounding boxes as the initial guess, FPFH (Fast Point Feature Histograms) features are computed for each point. These 33-dimensional vectors describe the local geometry around each point. RANSAC then finds correspondences between source and reference based on feature similarity, robustly handling large rotations (>90°) where ICP would diverge.

**Scale Extraction Post-ICP:** The 3×3 rotation block `A = T_final[:3, :3]` from the ICP result may contain a small residual scale factor `s_icp` if `with_scaling=True`:

```python
s_icp = np.mean(np.linalg.norm(A, axis=0))
R = A / s_icp
```

The column norms of the rotation matrix should all be 1.0 for a pure rotation. Their mean gives the scale factor. SVD is used to orthogonalise `R` and remove any numerical drift: `U, _, Vt = SVD(R); R = U @ Vt`.

Total scale = `init_s * s_icp`. This multiplicative composition is correct because the source was pre-scaled by `init_s` before ICP, so ICP's `s_icp` is relative to that.

---

## Section 3: PLY Rewriting — `apply_transformation_and_save()`

This is the most 3DGS-specific part of the code. A 3DGS PLY file has many custom attributes per vertex beyond just position. Here is what each one means and how it is transformed:

### 3.1 Position Transform (x, y, z)

```python
xyz = np.stack([plydata['vertex']['x'], ...], axis=1)   # shape (N, 3)
xyz_new = s * (xyz @ R.T) + t
```

The formula `y = s·R·x + t` is implemented as `s * (xyz @ R.T) + t`. Here `@ R.T` right-multiplies the row vectors of `xyz` by the transpose of R, which is equivalent to left-multiplying column vectors by R. Result is cast to `float32` since 3DGS PLY files use single precision.

### 3.2 Normal Transform (nx, ny, nz)

Normals are **direction vectors**, so they are only rotated — not scaled or translated (scaling a direction changes its length; translation makes no geometric sense for a direction):

```
normals_new = normals @ R.T
```

After rotation, normals are re-normalised to unit length using `np.linalg.norm`. The `np.where(norms == 0, 1.0, norms)` guard prevents division by zero.

### 3.3 Log-Scale Transform (scale_0, scale_1, scale_2)

Each 3DGS Gaussian has an ellipsoidal shape parameterised by three scale values. These are stored as **log-space values** (`scale_i = log(σᵢ)`) to allow unconstrained optimisation (scales must be positive, but logs can be any real number).

When we scale the entire scene by `s`, each physical scale `σᵢ` becomes `s · σᵢ`. In log space:

```
log(s · σᵢ) = log(s) + log(σᵢ) = log_s + scale_i
```

So the transform is simply:

```python
log_s = np.log(s)
plydata['vertex'][scale_name] += log_s    # for i in {0, 1, 2}
```

This addition in log-space is equivalent to multiplication in linear space. Note: only scale changes — not rotation or translation — affect the physical size of each Gaussian.

### 3.4 Orientation Transform (rot_0, rot_1, rot_2, rot_3)

Each Gaussian's orientation is stored as a unit quaternion `(rot_0, rot_1, rot_2, rot_3)` = `(w, x, y, z)`. To rotate the entire scene by `R`, we left-compose each Gaussian's quaternion with the rotation quaternion `q_R` corresponding to `R`:

```
q_new = q_R ⊗ q_old
```

This is computed via `multiply_quaternions(q_R, q_old)` (see §1.3).

`q_R` is obtained from `R` via `rotation_matrix_to_quaternion(R)` (see §1.2).

**Why not apply scale or translation to rotations?** Quaternions purely encode orientation — they are dimensionless unit vectors in $\mathbb{R}^4$. Scale and translation have no meaning for orientation.

### 3.5 File Write

```python
os.makedirs(os.path.dirname(os.path.abspath(output_ply_path)), exist_ok=True)
plydata.write(output_ply_path)
```

`plyfile` writes the modified `PlyData` object back to disk. Because `plydata` was read with all original attributes and only specific fields were modified in-place, every other attribute (spherical harmonics coefficients for view-dependent colour, opacity/alpha values, any extra custom fields) is preserved exactly.

---

## Section 4: Command Line Interface

### 4.1 `parse_points(pts_str)` — Point String Parser

Parses a string like `"1.0,2.0,3.0;4.0,5.0,6.0;7.0,8.0,9.0"` into a `(3, 3)` numpy array. Splits on `;` to get individual points, then on `,` to get coordinates. Used for `--ref_pts` and `--src_pts` in Umeyama mode.

---

### 4.2 `main()` — CLI and Mode Dispatcher

#### Negative Number Argument Fix

```python
new_argv = []
i = 1
while i < len(sys.argv):
    arg = sys.argv[i]
    if arg.startswith('--') and i + 1 < len(sys.argv):
        next_arg = sys.argv[i + 1]
        if next_arg.startswith('-') and len(next_arg) > 1 and (next_arg[1].isdigit()
            new_argv.append(f"{arg}={next_arg}")
```

`argparse` misinterprets arguments like `--ref_bbox -9.377` — it sees `-9.377` as another flag. This loop pre-processes `sys.argv` to convert those into `--ref_bbox=-9.377` (equals syntax), which `argparse` handles correctly.

#### Argument Registration

The parser registers:

- `--mode`: one of `{bbox, icp, umeyama, interactive, auto}`
- `--ref_ply` / `--src_ply` / `--output_ply`: file paths
- `--ref_bbox` / `--src_bbox`: lambda that splits a comma string into a float list
- `--voxel_size`: float, default 0.05 metres
- `--refine_scale`: boolean flag, enables scale refinement in ICP
- `--global_reg`: boolean flag, uses RANSAC global registration instead of bbox init
- `--ref_pts` / `--src_pts`: type `parse_points` — auto-parsed into numpy arrays

#### PLY Loading Strategy

Reference cloud (TLS): loaded via Open3D (`read_point_cloud`) which is memory-efficient for large TLS files. If Open3D fails, falls back to `plyfile`. Either way, `ref_pts` ends up as a `(N, 3)` numpy array.

Source cloud (3DGS): **always** loaded via `PlyData.read()` to preserve all attributes. `src_pts` is extracted separately for geometric operations.

---

## Section 5: The Five Alignment Modes

**Mode:** `bbox`

Computes initial bounding boxes (via cluster extraction if not provided), then calls `get_bbox_transform()` and directly writes the output. No rotation, just scale + translation. Fastest mode, least accurate. Good starting point or sanity check.

**Mode:** `icp`

1. Computes initial `(s, R, t)` from bounding boxes (or uses global registration if `--global_reg`).
2. Pre-scales source for ICP stability.
3. Runs Open3D ICP (Point-to-Plane or Point-to-Point with optional scaling).
4. Extracts final `(s, R, t)` from the ICP result matrix.
5. Saves aligned PLY.

Best for scenes where orientation is approximately known but fine alignment is needed.

**Mode:** `umeyama`

User provides `--ref_pts` and `--src_pts` from manual measurements (e.g., measuring a few branch tips or ground control points in both coordinate frames). Runs Umeyama SVD alignment on those N≥3 correspondences. Most accurate if correspondence points are well-chosen.

**Mode:** `interactive`

Wraps `umeyama` with a GUI. Opens Open3D viewer twice — once for TLS (cyan), once for 3DGS (yellow) — asking user to Shift+click 3 matching landmarks in each. Automatically runs Umeyama on the picked points.

**Mode:** `auto`

Fully automated pipeline combining all techniques:

1. Extract main cluster from both clouds (DBSCAN).
2. Compute bounding box scale and translation from clusters.
3. Translate both clouds to origin (around the reference centroid) for stable rotation.
4. Sweep 36 Z-axis rotations with ICP fitness scoring to find best orientation.
5. Compose the full initial transform matrix from all steps.
6. Run ICP refinement on the full point clouds.

The transform composition uses 4×4 homogeneous matrices:

```
T_total = T_from_origin · T_best_rot · T_to_origin · T_translation · T_scale
```

Matrix multiplication here correctly chains the transforms right-to-left: scale first, then translate, then bring to origin, then rotate, then bring back. The final `(s, R, t)` is extracted by decomposing the 3×3 block via column-norm scale extraction and SVD orthogonalisation (same method as in `run_icp_alignment`).

---

## Data Flow Summary

```
TLS PLY ──────────────────────────────────────> ref_pts (N×3)
                                                    |
3DGS PLY ──> src_ply_data (full attributes) ─> src_pts (M×3)
                                                    |
                        [Choose Mode] ──────> compute (s, R, t)
                                                    |
src_ply_data ──> apply_transformation_and_save(s, R, t)
                    ├─ xyz_new = s·R·xyz + t
                    ├─ normals_new = R·normals (normalised)
                    ├─ scale_i += log(s)
                    └─ rot_i = q_R ⊗ rot_i
                                                    |
                                          output aligned PLY
```

---

## Key Design Decisions

**Why `plyfile` instead of Open3D for writing?** Open3D only handles standard fields (xyz, rgb, normals). 3DGS PLY files contain ~60 custom fields per point. `plyfile` does in-place field modification and writes back the exact binary structure.

**Why log-space for scales?** It's how 3DGS training stores them — the neural renderer uses `exp(scale_i)` to get physical sizes. Adding `log(s)` in storage space is equivalent to multiplying by `s` in physical space.

**Why quaternion left-multiplication?** We want to rotate each Gaussian's orientation into world-space first, then apply the scene rotation. Left-multiplying by `q_R` appends the scene rotation on the outside, which is mathematically equivalent to `R · R_gaussian` in matrix form.

**Why voxel downsampling before ICP?** TLS clouds can have 50-100M points. ICP's nearest-neighbour search is O(N log N) per iteration. Voxel downsampling to `~voxel_size` spacing reduces N by 100-1000× while preserving the geometric shape. The final transform is applied to the full-resolution cloud.